PROFILE



염동섭

게임 클라이언트 프로그래머

C++ · Unreal Engine 5.4

CONTACT

dsyeom931008@gmail.com

010-7181-7394

GitHub 프로필 ↗

PROJECT STELLAR

UE5.4 기반 근접 액션 전투 시스템

구현 구조와 문제 해결·검증 사례



프로젝트 저장소 ↗

프로젝트 소스 코드와 README

기술 문서 ↗

기능별 구조와 설계 설명

프로젝트 구성

항목을 선택하면 해당 설명 페이지로 이동합니다. 관련 영상은 각 페이지에서 확인할 수 있습니다.

프로젝트 소개 03–04

프로젝트 개요 → 03

개발 환경과 담당 구현 범위를 소개합니다.

플레이 가능 범위 → 04

공격·방어와 다수 적 전투를 시연합니다.

문제 해결·검증 05–13

다수 AI 인식 지연 → 05–07

인식 지연의 원인과 개선 결과를 설명합니다.

후방 피격 후 타깃 유지 → 08–10

타깃 만료의 원인과 TTL 수정을 설명합니다.

가드 종료 중 피격 전환 → 11–13

가드 정리 오류의 원인과 수정을 설명합니다.

구현 구조 14–23

공격·피격 처리 → 14–15

공격 충돌부터 피격 반응까지 설명합니다.

실행 중단·반응 전환 → 16–17

기존 실행 정리와 새 반응 시작을 설명합니다.

패링 판정·결과 전달 → 18–19

패링 판정과 양측의 후속 처리를 설명합니다.

전투 참여 역할 배정 → 20–21

여러 적의 역할 배정과 행동 결정을 설명합니다.

디버그 오버레이 → 22–23

전투 상태를 관찰하는 도구 구조를 소개합니다.

프로젝트 개요

가드·패링·회피와 다수 적 전투를 구현한 UE5.4 기반 근접 액션 프로젝트입니다.

개발 기간

2025.12.01 ~ 현재

2026.10.21 기준

개발 규모

1인 개발 · AI 협업

개발 환경

Unreal Engine 5.4

C++ / Blueprint

프로젝트 범위

전투·AI·피드백 구현

에셋 설정·게임플레이 통합

핵심 구현 범위

01

전투 처리와 실행 관리

피해·방어 판정과 액션·리액션의
실행 결정, 중단 및 전환을 처리합니다.

공격·피격·가드·패링·회피

02

AI 인지와 전투 참여

인지·피격 근거에 따라 대상을 판단하고
교전·경계·관찰 역할을 배정합니다.

대상 판단 · 행동 선택 · 참여 역할

03

전투 피드백과 검증

실행 시점에 시각·음향 피드백을 연결하고
디버그 오버레이로 상태를 확인합니다.

전투 연출 · HUD · 디버그 도구

외부 에셋 활용 범위

캐릭터·모션·무기·환경·VFX·효과음 활용
폰트·아이콘의 출처와 제작 방식 구분

외부 에셋 출처 보기 ↗

구현 범위·AI 협업

전투·AI·피드백·HUD·디버그 도구 구현
외부 에셋 가공·설정 및 게임플레이 통합
본인: 요구·검토 / Codex: 구현·검증 지원

구현 구조 상세 보기 ↗

실제 플레이로 보는 전투

공격과 방어를 연결한 반격, 균형 붕괴 후 처형, 다수 적과의 교전을 구현했습니다.



다수 적 전투 시연 ↗

여러 적의 교전 참여와 실제 공격을 확인할 수 있습니다.

공격과 방어

콤보 공격과 가드·패링·회피로
상대의 공격에 대응합니다.

반격과 처형

회피 후 카운터 러시로 반격하고,
균형이 무너진 적을 처형합니다.

다수 적과의 교전

여러 적이 접근하는 상황에서
교전 참여와 주변 경계가 이어집니다.

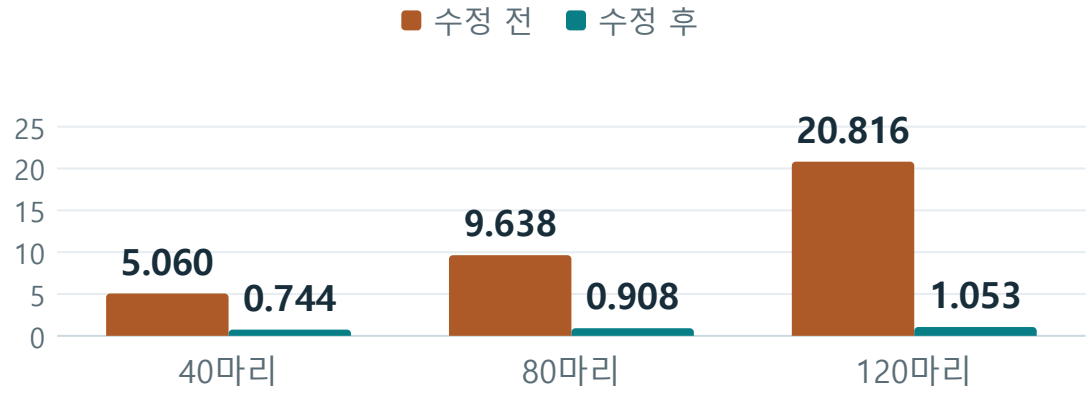
관련 기능 시연

회피·반격 ↗ 균형·처형 ↗ 무기·피드백 ↗

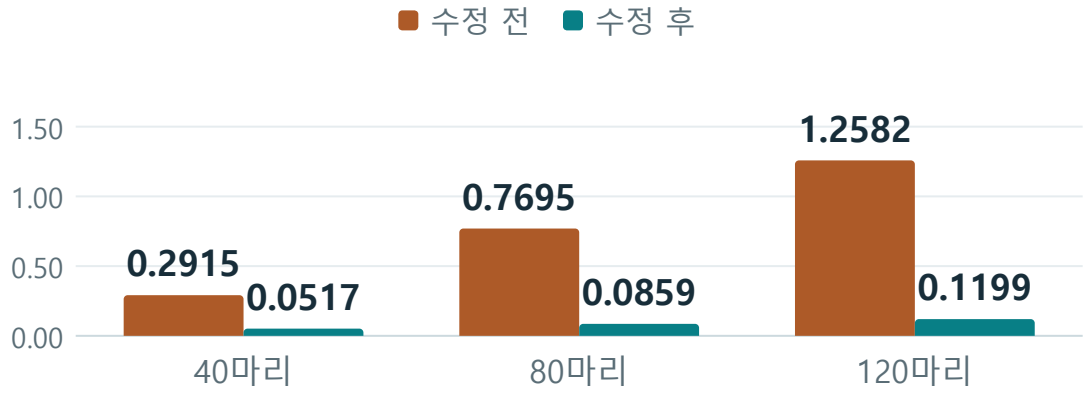
문제: 다수의 적이 배치된 환경에서 대상 인식 지연

플레이어를 감지해 대응하도록 구성했지만, 적 수가 늘어날수록 최초 인식이 늦어졌습니다.
40·80·120마리에서 인식 로그와 프로파일링 CSV를 수집해, 인식 지연과 감지 처리 비용을 비교했습니다.

최초 유효 대상 인식 지연 · p95 (초)



AIPerception 처리 시간 · p95 (ms)



최초 유효 대상 인식 지연 | AI별 계측 시작부터 플레이어의 첫 성공 감지 콜백까지 걸린 시간

AI Perception 처리 시간 | 게임 스레드의 AI Perception 처리에 사용한 프레임별 시간

측정 조건 | 40·80·120마리 · 전후 각 3회 · 회당 40초 · 동일 배치·카메라 · 녹화 없이 측정

측정 환경 | UE 5.4.4 · Win 11 · Ryzen 9 7950X3D · RTX 4070 Ti · D3D12 · Development

집계 방식 | 실행별 p95의 3회 중앙값
반복 범위·정상 인식은 7쪽

40~120마리에서 최초 인식 지연을 약 0.74~1.05초로 단축

인식 지연·감지 처리 비용 감소. 전체 프레임 시간 개선은 확인되지 않았습니다.

측정: 2026.10.02 · p95: 95백분위수 · 영상 13은 측정과 별도의 동작 비교 시연

해결: 같은 편 Enemy를 감지 후보에서 제외

같은 편 Enemy도 Sight 감지를 거친 뒤 제외되어, 불필요한 감지 처리가 발생했습니다.
Player는 적대, Enemy는 아군으로 분류해 같은 편을 감지 단계에서 제외했습니다.

원인 관찰: 감지 후 제외되는 같은 편 Enemy

120마리 · AI별 관측값의 p95 · 3회 중앙값

	누적 감지 객체 수	같은 편 Enemy	최대 등록 타깃 수
수정 전	111	110	1
수정 후	1	0	1

원인: 같은 편까지 감지한 뒤 제외

	Sight 감지		감지 콜백		전투 타깃 등록
수정 전	Player + Enemy	→	**Enemy 제외 (Provider 검사)**	→	Player만 등록
수정 후	팀 관계로 Enemy 감지 제외	→	**Player만 전달**	→	Player만 등록

수정: 팀 관계를 명시해 Sight 필터 적용

ACAIController::
GetTeamAttitudeTowards(...) const

```
if (Other.IsA<ACPlayer>())
    return ETeamAttitude::Hostile;
if (Other.IsA<ACEnemy>())
    return ETeamAttitude::Friendly;
```

Sight: 적대 감지 ON · 아군/중립 OFF

수정 핵심: 감지 단계에서 같은 편 제외

같은 편 Enemy가 감지 콜백으로 넘어오기 전에 제외합니다.
Player의 전투 타깃 등록 흐름은 유지합니다.

변경 효과

전투 대상이 될 수 없는 후보의
불필요한 감지·콜백 처리 감소

검증: 반복 측정에서도 지연 감소·전원 인식 완료

수정 전후 각각 3회씩 측정해 모든 AI의 플레이어 인식 완료를 확인했습니다.
수정 후 인식 지연의 반복 측정 범위와 전체 프레임 시간도 함께 확인했습니다.

수정 후 인식 p95 · 반복 측정 범위

개체 수	중앙값	3회 최소–최대	전후 매회 완료
40마리	0.744초	**0.744–0.745초**	40 / 40
80마리	0.908초	**0.904–0.930초**	80 / 80
120마리	1.053초	**1.049–1.067초**	120 / 120

측정 기준: 40·80·120마리 / 전후 각 3회 / 각 40초 / 녹화 없이 측정

정상 인식과 감지 처리 비용

등록 타깃 수

수정 전		수정 후
1	→	1

AI Perception 처리 시간

수정 전		수정 후
1.2582 ms	→	0.1199 ms

120마리 · 실행별 p95의 3회 중앙값

검증 결론

3회 모두 플레이어 인식 완료
수정 후 지연의 반복 측정 범위 확인

비교·해석 조건

p95는 각 실행에서 집계
개별 값은 검증 상세 참고

검증 범위·제한

전체 프레임 p95 개선 없음
20.7365 → 21.1009 ms

문제: 후방 피격 후 공격자를 추적하지 못함

후방에서 피격된 적이 리액션을 마친 뒤에도 공격자를 향해 돌아보지 않았습니다.
공격자 타깃 유지 여부와 종료 후 추적 회전을 같은 환경에서 비교했습니다.

수정 전 · 기존 정책 재현



리액션 종료 후 · 회전 없음

피격 근거 만료

전투 참여 링 없음

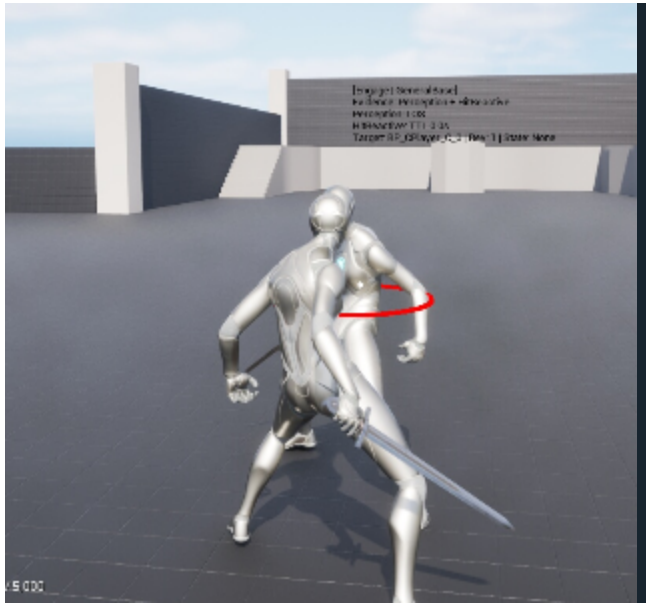
동일 실행 · 종료 직후 로그

Reaction=None

Target=None

Policy=NoCombatTarget

수정 후 · 변경 정책



리액션 종료 후 · 추적 회전

공격자를 향해 회전

전투 참여 링 유지

동일 실행 · 종료 직후 로그

Reaction=None

Target=BP_CPlayer_C_0

Policy=Tracking

피격 리액션 종료 시 타깃 없음

TTL이 먼저 만료되어 타깃 해제
후속 Investigate는 비교에서 제외

피격 리액션 종료 시 타깃 유지

종료 직후 타깃과 포커스를 적용하고
공격자를 향해 회전하는 것을 확인

동일 에셋·TTL(피격 근거 유지 시간) 1초 · 10월 1일 보존 환경 재현 · 후속 Investigate 제외

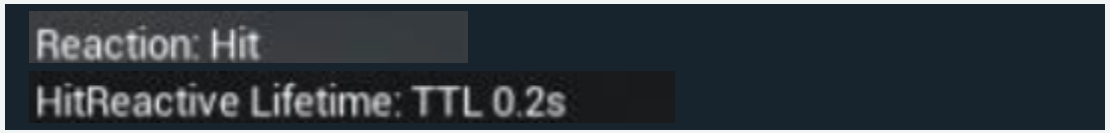
해결: 피격 리액션 종료 후 TTL 감소 시작

피격 근거의 유지 시간(TTL)이 리액션 중 소모되어, 종료 시 공격자 타깃이 사라졌습니다.
근거 등록은 피격 수락 시 유지하고, TTL 감소 시작을 해당 리액션 종료 이후로 옮겼습니다.

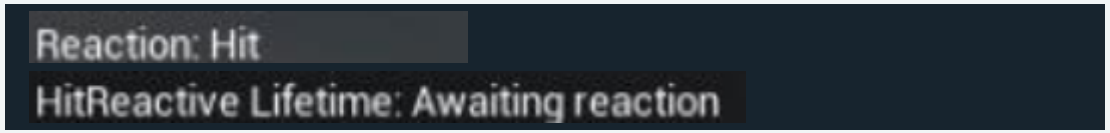
관찰: 피격 진행 중 TTL 상태

재촬영 진단 오버레이 발췌

수정 전 · 피격 중 TTL 감소



수정 후 · 피격 종료까지 TTL 시작 대기



원인: 리액션 종료 전에 피격 근거의 TTL 소진

	피격 시작	피격 진행 중	리액션 종료 추적 연결
수정 전	TTL 시작 →	TTL 만료 타깃 해제 →	**TTL 만료 타깃 없음**
수정 후	타깃 등록 TTL 시작 대기 →	타깃 유지 TTL 시작 대기 →	**TTL 시작 추적 연결**

수정: 피격이 끝난 뒤 TTL 시작

① 적의 **피격 리액션 종료** 확인

② 해당 피격 당시 **저장한 공격자** 확인
ResultSerial로 피격과 공격자를 연결

③ 해당 공격자에 대한 **HitReactive TTL 시작**

```
CombatParticipationComp_Injected->
    StartHitReactiveEvidencePostReactionTTL(
        combatantTarget->Get(), resultSerial);
```

수정 핵심: 근거 등록과 TTL 시작 분리

피격 근거는 수락 시 등록하고,
TTL은 해당 리액션 종료 후 시작합니다.

연결 기준

ResultSerial로 피격·공격자 연결
연속 피격·종료 순서는 10쪽에서 검증

검증: 늦거나 중복된 종료 알림에도 기록이 제때 만료되는지 확인

적은 자신을 때린 공격자를 기억해 타깃 판단에 사용합니다. 이 기록은 마지막 피격 반응 종료 후 1초간 유지됩니다.
연속으로 맞았을 때 이전 종료 알림이 늦거나 중복해서 도착해도, 기록의 유지 시간이 늘어나지 않는지 확인했습니다.

① 같은 공격자에게 두 번 연속으로 맞은 경우

아래 세 행은 0.50초 → 0.70초 → 1.60초 순서로 이어지는 하나의 검사입니다.

시점	어떤 상황을 만들었나?	무엇을 확인했나?
0.50초	두 번째 피격 반응을 종료했습니다.	**1초 뒤인 1.50초에 기록이 만료되도록 설정됐습니다.**
0.70초	첫 번째 피격의 종료 알림을 뒤늦게 전달하고, 두 번째 피격의 종료 알림도 다시 전달했습니다.	**만료 시각은 1.50초 그대로였습니다. 1.70초로 늘어나지 않았습니다.**
1.60초	만료 시각이 지난 뒤 기록을 확인했습니다.	**해당 공격자에게 맞았다는 기록이 만료됐습니다.**

② 서로 다른 공격자에게 맞은 경우 · 별도 검사

A의 기록이 만료돼도, 아직 시간이 남은 B의 기록은 유지돼야 합니다.

1.25초	A 기록은 1.20초, B 기록은 1.50초에 만료되도록 설정한 뒤 확인했습니다.	**A의 기록만 만료되고, B의 기록은 유지됐습니다.**

자동 검사와 실제 시연

자동 검사 · NullRHI

4그룹 · 19시나리오 모두 통과

이전 종료·요청 거절
중단·공격자별 독립 만료

실제 시연

피격 반응 후 대상 추적 복귀

Tracking 상태 전환 확인

기록 유지와 현재 타깃 선택은 별개

검증 결론

늦거나 중복된 종료에도 만료 시각 유지
공격자별 기록은 각자의 시각에 만료

비교·해석 조건

수정 전은 정책 복원 시연
당시 바이너리를 사용하지 않음

검증 범위·제한

공격자 소멸 후 대기 기록
정리 보완안 원본 미반영

문제: 가드 종료 중 피격 모션·이펙트 누락

가드 해제 중에도 피격 반응이 이어져야 했지만, 피해만 적용되고 모션·이펙트가 누락됐습니다.
실행 화면과 로그를 대조해, 피해 적용 후 피격 반응이 시작되지 않는 구간을 확인했습니다.

수정 전



화면 관찰

① 피격 모션 전환 없음

피격 반응 효과 없음

실행 로그 · 핵심 필드 발췌

Damage=20.000
Stage=InterruptGuardOut

[ReactionDecision]

Overlay handling failed.

수정 후



화면 관찰

① 피격 모션 전환

② 피격 반응 효과 표시

실행 로그 · 핵심 필드 발췌

Damage=20.000
Stage=InterruptGuardOut

보조 근거 · 실행 로그

동일 실패 로그 없음

피해 적용 후 피격 모션·이펙트 누락

가드 중단까지 진행했으나
피격 리액션 시작 전 실패

피해 적용 후 피격 모션·이펙트 재생

가드 중단 후 피격 리액션으로 전환
피격 모션과 이펙트 재생 확인

① 피격 자세 / ② 피격 반응 효과 · 검 궤적과 구분 · 연속 영상에서 확인: 영상 11 · 00:05–00:23

해결: 가드 상태 정리가 끝났어도 피격 전환 허용

가드 중단과 피격 시작 준비에서 가드 상태 정리가 연속으로 요청됐습니다.
이미 정리된 경우에도 요청을 수락하도록 수정해 피격 전환이 중단되지 않게 했습니다.

호출 흐름과 실패 지점

① 기존 가드 중단 처리
ClearGuardState 완료

↓

② 새 피격 리액션 시작 준비
ClearGuardState 재요청

수정 전 · 요청 거절 (false)
가드 상태가 이미 없음
Hit 시작 전 처리 종료

수정 후 · 요청 수락 (true)
이미 정리된 상태도 허용
Hit 시작 단계로 진행

실제 실패 로그 · 타임스탬프 생략

```
[OverlayHandling] Failed Handling=
EObservableOverlayHandling::ClearGuardState
```

수정한 수락 조건

UCDefenseComponent::
CanApplyOverlayHandling(...) const

```
// ...
case EObservableOverlayHandling::
    ClearGuardState:
- return HasGuardRuntimeState();
+ return true;
// ...  관련 분기 발췌   - 수정 전 / + 수정 후
```

변경 범위: 정리 요청의 수락 조건

가드 상태 유무로 요청을 거절하던 분기를 수정했습니다.
실제 상태 제거 호출은 유지합니다.

수정 핵심: 이미 정리된 상태도 정상 처리

같은 정리 요청이 다시 들어와도
새 피격 반응의 시작을 막지 않도록 합니다.

적용 범위

ClearGuardState 정리 요청
다른 실행 허용 조건은 별도 판단

검증: 가드 종료 중 피격 후 다음 가드까지 실행

가드를 내리는 도중 공격을 맞았을 때, 가드 정리 과정에서 막히지 않고 피격 반응이 실행되는지 확인했습니다.
피격 반응 종료 후 기본 이동 속도 설정이 복원되고, 다시 가드를 시작·유지·종료할 수 있는지 확인했습니다.

실제 실행: 하나의 연속된 검사

모든 시간은 처음 가드 해제를 요청한 순간부터 계산했습니다.

경과 시간	입력·발생한 일	확인한 결과
0.000초	가드 해제를 요청했습니다.	**가드를 내리는 종료 동작이 시작됐습니다.**
0.076초	가드를 내리는 도중 적의 공격에 맞았습니다.	**남은 가드 정리 요청 수락 후 피격 반응 실행** **HP 5,000 → 4,700**
1.292초	피격 반응 종료와 속도 설정 확인 후, 다시 가드를 요청했습니다.	**기본 이동 속도 설정 600 확인** **새 가드 시작 동작 실행**
2.806초	새 가드의 유지 상태를 확인한 뒤, 종료 동작으로 전환됐습니다.	**가드 유지 상태 진입 확인** **이어서 가드 종료 동작 시작**
4.814초	두 번째 가드가 끝난 뒤 상태를 확인했습니다.	**기본 이동 속도 설정이** **다시 600인지 확인했습니다.**

실제 실행과 별도 자동 검사

실제 실행 · 격리 PIE

피격 반응 → 다음 가드
시작·유지·종료 확인

Run 600 속도 설정 복원 확인
달리는 동작의 재개 검사는 아님

별도 자동 검사

24조건 × 25회
정리 호출 총 600회

각 조건 종료 시 가드 상태 해제
기본 이동 속도 설정 복원 확인

요청은 재현 코드에서 실행 · 격리 PIE | 이동 속도: PIE Run 600 / 자동 검사 Run 420·Sprint 620 | 자동 조건: 가드 상태 4 × 정리 경로 3 × 이동 설정 2

검증 결론

피격 반응·기본 이동 속도 설정 복원 확인
다음 가드와 반복 정리 후 상태 복원 확인

비교·해석 조건

기존 시연 피해 20 · 본 검증 300
자동 검사와 PIE 이동 설정 구분

검증 범위·제한

Sprint 유지·800 복원 미확인
수동 PIE 이펙트 동등성 미확인

공격 실행과 피격 반응의 연결

Player의 일반 공격이 Enemy에 적중한 장면입니다.
공격 실행, 충돌과 결과 판정, 피격 반응의 세 단계로 구성됩니다.



1 공격 실행

Player의 입력에 따른
공격 동작 실행

2 충돌과 결과 판정

무기 충돌 데이터 전달 후
Enemy의 피해·방어 결과 판정

3 피격 반응

판정 결과에 따른
Enemy의 피격 반응

공격자는 공격 실행과 충돌 데이터 전달을, 피격자는 결과 판정과 반응을 담당

영상 01 · 00:09–00:31 | 공격 실행 → 피해 적용 → 피격 반응

공격·피격 처리의 책임과 실행 흐름

일반 타격은 공격 실행(Action), 충돌·결과 판정, 피격 반응(Reaction)의 순서로 처리됩니다.
공격자는 충돌 데이터를 전달하고, 피격자는 자신의 상태로 피해·방어 결과와 반응을 결정합니다.



일반 타격 · Player → Enemy

설계 이유

피해 적용과 피격 반응은 별개의 판단

피해 적용 여부와 피해량은 피격자의 방어 상태와 공격 속성으로 결정합니다. 피격 반응은 현재 실행 중인 액션·리액션과 전환 조건에 따라 실행 여부를 결정합니다.

① 공격 실행 · 왼쪽부터 →

공격자 Player
행동 판단
현재 상태 기반 정책 평가 요청
ActionOrchestrator

➡

공격자 Player
실행 관리
현재 실행 관리·판단 결과 적용
ActionComponent

➡

공격자 Player
공격 실행
Montage 재생·종료 정리
Action

Notify로 무기 충돌 활성화 ⬇

② 충돌·결과 판정 · ← 오른쪽부터

공격자 Player
무기 충돌
공격 판정 구간 내 충돌 발생

⬅

공격자 Player
충돌 검증·전달
공격 판정 구간·중복 충돌 확인
CombatSignalSource

⬅

피격자 Enemy
피해·방어 판정
피격자 상태로 결과 결정
CombatSignalTarget

⬇ 판정 결과로 반응 요청

③ 피격 반응 · 왼쪽부터 →

피격자 Enemy
반응 판단
현재 상태 기반 정책 평가 요청
ReactionOrchestrator

➡

피격자 Enemy
실행 관리
현재 반응 관리·판단 결과 적용
ReactionComponent

➡

피격자 Enemy
피격 반응
Montage 재생·종료 정리
Reaction

공격 중 피격에 따른 실행 전환

Enemy가 공격 도중 Player의 타격을 받아 피격 자세로 전환하는 장면입니다.
공격 진행, 피격에 따른 공격 중단, 피격 반응 시작의 세 단계로 구성됩니다.

1 공격 진행



Enemy가 진행 중인 공격 동작

Enemy HP 200 / 200

2 피격·공격 중단



Player의 타격으로 Enemy 공격 중단

Enemy HP 195 / 200

3 피격 반응 시작



Enemy가 공격에서 피격 자세로 전환

Enemy HP 195 / 200

중단이 허용된 공격은 기존 실행을 정리한 뒤 피격 반응으로 전환

영상 02 · 00:05–00:28 | 은색 Enemy의 공격·피격 상태와 종료 후 복귀 확인

실행 중단 조건과 피격 반응의 시작 순서

공격 중 피격된 Enemy가 실행 중인 공격을 중단하고 피격 반응으로 전환하는 구조입니다.
반응의 중단 요구와 공격의 중단 허용을 함께 확인하고, 기존 공격을 정리한 뒤 새 반응을 시작합니다.



사례 · 공격 중 피격된 Enemy

설계 이유

이전 공격 상태가 새 반응에 남지 않게 정리

피격 반응 중에 이전 공격의 충돌 판정이 남지 않도록, 충돌·트레일과 실행 정보를 정리한 뒤 반응을 시작합니다. 개별 피드백은 각자의 종료 기준에 따라 따로 종료합니다.

① 중단 조건 확인·요청 · 왼쪽부터 →

피격 반응의 중단 요구
현재 공격의 중단을 요구하는 반응인지 확인
Reaction.WantIntervention

→

현재 공격의 중단 허용
해당 피격 반응에 의한 중단 허용 여부·시점 확인
Action.AllowIntervention

→

공격 중단 요청
두 조건이 충족되면 현재 실행 중인 공격에 요청
ReactionOrchestrator / Component

↓

② 실행 중인 공격 정리 · ← 오른쪽부터

Montage 재생 중지
실행 중인 공격의 Montage 재생 중지
StopMontage

←

무기 충돌·트레일 정리
무기 충돌·트레일 비활성화
무기 전환·충돌 문맥 정리
CleanupRuntimeEffects

←

공격 실행 정보 초기화
공격 데이터·허용 구간 초기화
컴포넌트에 종료 알림
ClearRuntime / ActionComponent

↓

③ 새 피격 반응 시작

기존 공격의 정리가 끝나면 Hit Reaction과 Montage 실행

ReactionComponent / Reaction.Start

패링의 피해 차단·반응·균형 변화

Player가 Enemy의 공격을 패링한 장면입니다.
충돌 데이터 전달, 방어 결과 판정, 양측 후속 처리의 세 단계로 구성됩니다.



Player HP 유지

패링으로 공격을 막아 HP 유지

Player 패링 반응

판정 결과에 따른 패링 반응

Enemy의 남은 균형 감소

패링이 누적될 때마다
남은 균형이 1씩 감소
0이 되면 균형 붕괴 반응 요청

패링은 HP 피해를 차단하고, Player의 반응과 Enemy의 균형 변화로 이어짐

영상 03 · 00:09–00:34 | 사진: 균형 2→1 / 영상: 3→2 · 처리 규칙은 코드에 근거.

패링 판정과 양측 후속 처리

Player가 Enemy의 공격을 패링했을 때의 처리 구조입니다.
충돌 데이터를 받아 방어 결과를 판정하고, Player의 피해·반응과 Enemy의 남은 균형에 반영합니다.

① 충돌 발생

공격자 Enemy

공격 충돌 발생

공격 구간·중복 충돌 확인
Player에 충돌 데이터 전달

CombatSignalSource

설계 이유

피해가 없어도 후속 처리는 필요

방어자 반응을 요청하고 공격자 균형을 줄입니다. 반응 실행 여부는 현재 실행 상태·전환 조건으로 결정합니다.

② 피격 조건과 방어 결과 판정

피격자 Player

충돌 데이터 수신
TakeDamage로 충돌 데이터 수신

피격 조건 확인
생존·피격 허용 상태·공격 속성 확인

방어 결과 결정
회피·패링·가드 조건 평가

이 사례의 판정: Parry

타격 수락 · 최종 피해량 0

CombatSignalTarget / Defense

③ 판정 결과의 양측 반영

Player · 방어자 처리

피해 처리
HP 차감 없음
피해량 0

패링 반응 요청
정책 허용 시 실행
ReactionOrchestrator

자신의 피해 확정 후 패링 반응 요청

Enemy · 전달받은 결과 처리

남은 균형 감소

패링 누적 횟수 +1 · 남은 균형 -1

CombatResultPacket / Balance

같은 대상을 둔 다수 적의 전투 역할

여러 Enemy가 같은 Player를 대상으로 서로 다른 참여 역할을 배정받은 장면입니다.
참여 근거 수집, 참여 역할 배정, 개별 행동 결정의 세 단계로 구성됩니다.



Engage · 교전 3명

기본 2명 + 피격 근거 추가 1명

- 피격 근거 없는 Engage
- 인식·피격 근거가 있는 Engage
- 피격 근거만 있는 Engage

Alert · 경계 1명

거리 조건에 따라 접근·경계

Observe · 관찰 1명

대상을 관찰하는 역할

None · 미배정

현재 참여 역할 없음

인식·피격 근거와 참여 상한에 따라, 같은 대상을 둔 적들의 역할을 배정

영상 06 · 00:33–01:25 | 사진: 기본 2+추가 1·근거별 링 / 영상: 기본 1+추가 1·역할 표시

참여 근거 수집과 역할 배정·행동 결정

참여 관리 Subsystem은 인식·피격 근거를 모아 대상별 참여 상한 안에서 역할을 배정합니다.
개별 AI는 현재 실행 상태를 먼저 확인하고, 배정 역할과 거리에 따라 행동을 결정합니다.



장면의 역할 배정

	배정 / 상한
교전 Engage	3 / 3명
경계 Alert	1 / 1명
관찰 Observe	1 / 1명

사진: 기본 2+추가 1 / 영상 06: 1+1

설계 이유

참여와 행동 판단 분리

참여 인원을 대상별로 제한하되, 역할만으로 행동을 정하지 않고 각 AI의 거리·실행 상태에 맞춰 추적·공격 등을 결정합니다.

① 근거 수집·후보 구성 · 왼쪽부터 →

참여 근거 수집
시야·기억: Perception
직접 피격: HitReactive
가드·패링 결과도 피격 근거에 포함

→

참여 근거 병합
같은 Enemy·대상 조합의 인식·피격 근거를 통합

→

참여 후보 구성
유효 근거·대상 유효성 확인
대상 우선순위·거리로 정렬

② 대상별 역할 배정 · ← 오른쪽부터

기존 역할 유지
유효 근거·배정 유지 조건 확인

←

승격·신규 배정
역할별 빈자리와 근거로 배정

←

배정 결과 전달
대상·역할·기본/추가 참여 전달

배정 개념: 기본 참여 우선. 기본 상한 이후에는 피격 근거가 있는 후보만 추가 배정.

③ 역할별 행동 선택

현재 실행 상태를 먼저 확인한 뒤, 배정 역할과 거리로 행동 선택

Engage · 교전 / 추격
멀면 추적, 가까워지면 공격 조건에 따라 공격

Alert · 경계 / 추격
멀면 추적, 가까워지면 경계 행동 수행

Observe · 대상 관찰
공격에 참여하지 않고 대상을 관찰

화면 패널과 월드 도형으로 확인하는 전투 상태

캐릭터의 상태·처리 기록과 전투 공간의 역할·방향·범위를 함께 확인하는 디버그 도구입니다.
캐릭터 상세·이벤트 로그·월드 요약의 화면 패널과 월드 디버그 드로우로 구성됩니다.



① 캐릭터 상세

Player·관찰 Enemy의
현재 상태와 최근 판단 정보

② 이벤트 로그

실행 판단·전투 판정·AI 처리 등
발생한 이벤트의 이력

③ 월드 요약

최근 주요 처리 결과와
대상별 전투 참여 현황

④ 월드 디버그 드로우

전투 참여 역할·공격 방향·앵커 등
공간 정보를 도형으로 확인

상태·처리 이력은 패널로, 역할·방향·범위는 월드 도형으로 확인

영상 10 · 00:13–01:07 | 사진은 참여 링·텍스트. 방향·앵커 도형은 해당 기록·표시 조건에서 출력.

관찰 데이터를 화면 패널과 월드 도형으로 출력

컴포넌트·Subsystem의 현재 상태와 이벤트·진단 기록을 관찰 데이터로 사용합니다.
화면 패널은 텍스트를 구성해 출력하고, 월드 도구는 위치·방향·범위를 도형으로 그립니다.



화면 패널과 월드 디버그 드로우

설계 이유

현재 상태와 사건 기록을 함께 확인

현재 상태만으로는 과거 판단의 이유를 알기 어려워 당시의 정보를 따로 기록합니다.
피격 반응과 대상·TTL을 대조해 반응 중 대상이 만료된 상황을 확인했습니다. (8–10쪽 사례)

HUD · 디버그 출력의 진입점

관찰 대상을 확인하고 상태를 조회한 뒤, 화면 패널과 월드 드로우를 각각 호출

관찰 데이터

현재 상태 조회

실행·역할·위치 정보

Component / Subsystem

이벤트·진단 기록

판단 시점의 정보 보관
현재 상태와 구분해 조회

SnapshotStore

출력 A · 화면 패널

정보·텍스트 구성

현재 상태·이벤트·요약 구성

ViewDataBuilder / TextFormatter

→

패널 그리기

화면 배치·텍스트 출력

CanvasRenderer

출력 B · 월드 디버그 드로우

기능별 공간 정보 구성

역할·좌표·방향·반경

기능별 Debug Helper

→

월드 도형 그리기

링·화살표·범위·텍스트

DrawDebug Helpers

월드 표시 예: 참여 역할 링 · 피격 근거 앵커의 기준점·범위 · Attack Facing의 방향·보정 범위